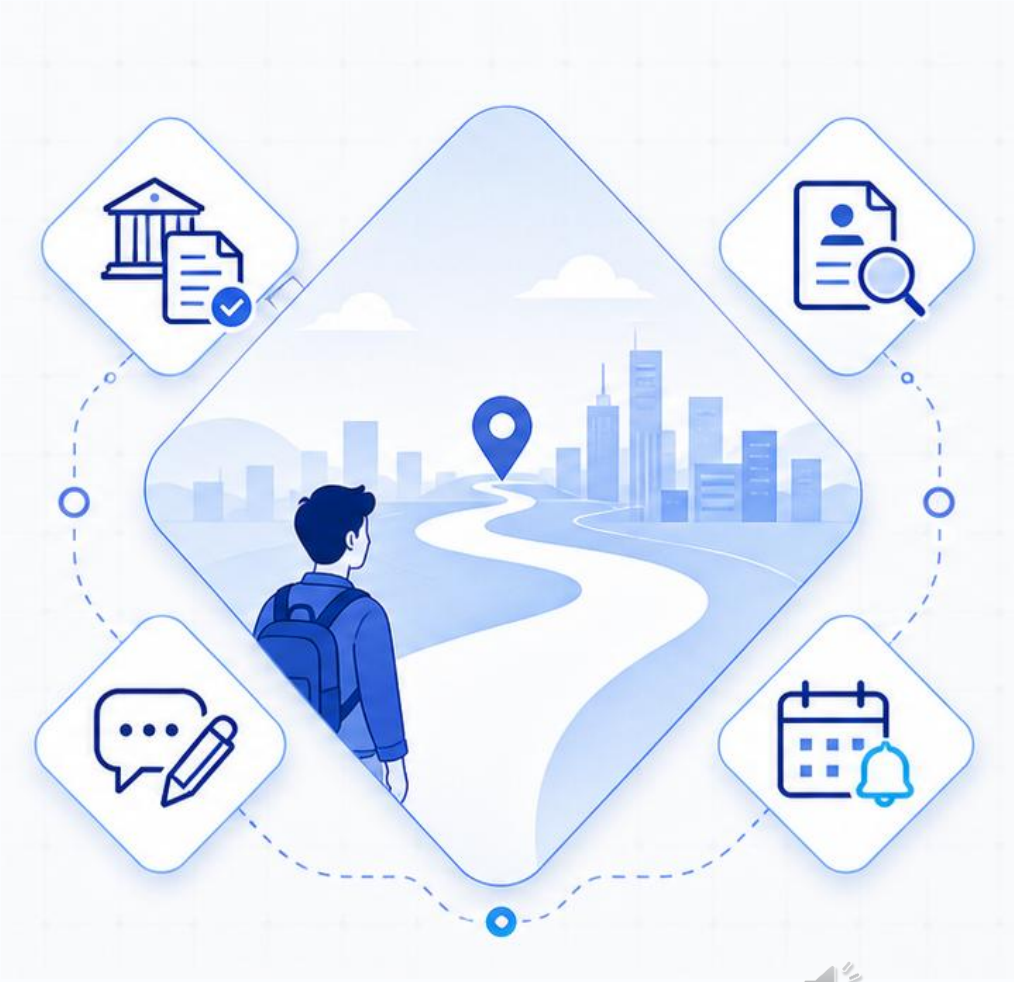
YouthPath

Total Assistant for Youth Career Entry



Deep Learning Methods and Applications | Team 10
2021016453 | 강정현
2021057401 | 김희승
2021054057 | 조재원
2021098040 | 홍성래
2023067501 | 박채완

Contents

01. Problem Definition and Motivation

02. Background and related work

03. System design and technical approach

04. Dataset and experiment plan

05. Feasibility and team planning

Problem Definition and Motivation

Problems

NEED

Cognitive · Technical

Policy Decoding Safety-net
Translating personal experience

NEED

Environmental · Effort

Searching scattered platforms
Tracking discontinuous deadlines

PAIN A

Information barrier

- **Interpretation bottleneck**

  Policy sites only list programs — youth must read dense PDFs and self-match eligibility in admin jargon

- **Data fragmentation**

  General LLMs lack live filings (cutoff); platforms don't link resumes with policy or hiring context

PAIN B

Execution barrier

- **Search-cost overload**

  Hopping across 5–6 fragmented platforms drains energy that should go into actual job preparation

- **Deadline-management risk**

  Staggered application windows mean a few days' lag can cost millions in subsidies or a job opportunity

Real-world relevance & Justification

Personal Efficiency

Frees youth to focus on self-development and substantive job preparation

Information time&energy saved

Technical Reliability

Grounded in public data and corporate filings — guidance with no factual drift

RAG + MCP → hallucination-free guidance

Macroscopic Impact

Reduces welfare blind spots, lifts policy uptake and budget effectiveness

Closes gap → No youth left behind

Background & Relative Work : Models & Tools

Large Language Models

- **LUXIA:**
  Korean - specialized
- **GPT-4o-mini:**
  Response Evaluation
- **Gemini 2.0 Flash:**
  Handling long-context

RAG

- **Static RAG:**
  Retrieves pre-stored policy and resume datasets
- **Dynamic RAG:**
  Retrieves real-time business reports from DART
- **Limitations of Traditional LLM:**
  Hallucination, Lack of specificity, Outdated knowledge
- **Advantages of RAG:**
  Improving factual accuracy, more specific and context-aware respon ses, Keeping information up-to-date without retraining the model

MCP

- Standardized interface for connecting external APIs and tools
- **Limitations of Traditional LLM:**
  1. Cannot access real-time external APIs or databases
  2. Cannot directly interact with internal systems or tools
  3. Cannot directly call APIs
- **Advantages of MCP:**
  Enables standardized access to external APIs and tools

Multi-Agent(LangGraph)

- **Router Agent:**
  analyzes user intent and assigns tasks
- **Sub-Agents:**
  Policy Agent, Job Agent, Resume/Interview Agent, Calendar Agent
- Each agent can run independently, while results are integrated around the user goal

Background & Relative Work : Improvement over Existing Services

Existing Services	YouthPath Improvement
Government24	Provides personalized policy recommendations based on detailed user inputs Clearly identifies eligibility and suggests application strategies
ChatGPT	Uses RAG and MCP for real-time retrieval and updated information
JOBKOREA	Provides resume writing guidelines based on company reports and job postings
Personal Calendar	Automatically synchronizes policy application deadlines and sends reminders

Our Solution: The Integrated YouthPath Experience

1. Policy

Interprets eligibility rules
and explains likely application fit

2. Job

Recommends postings
by role, region, and experience

User Profile

major · experience ·
region · target role

3. Resume

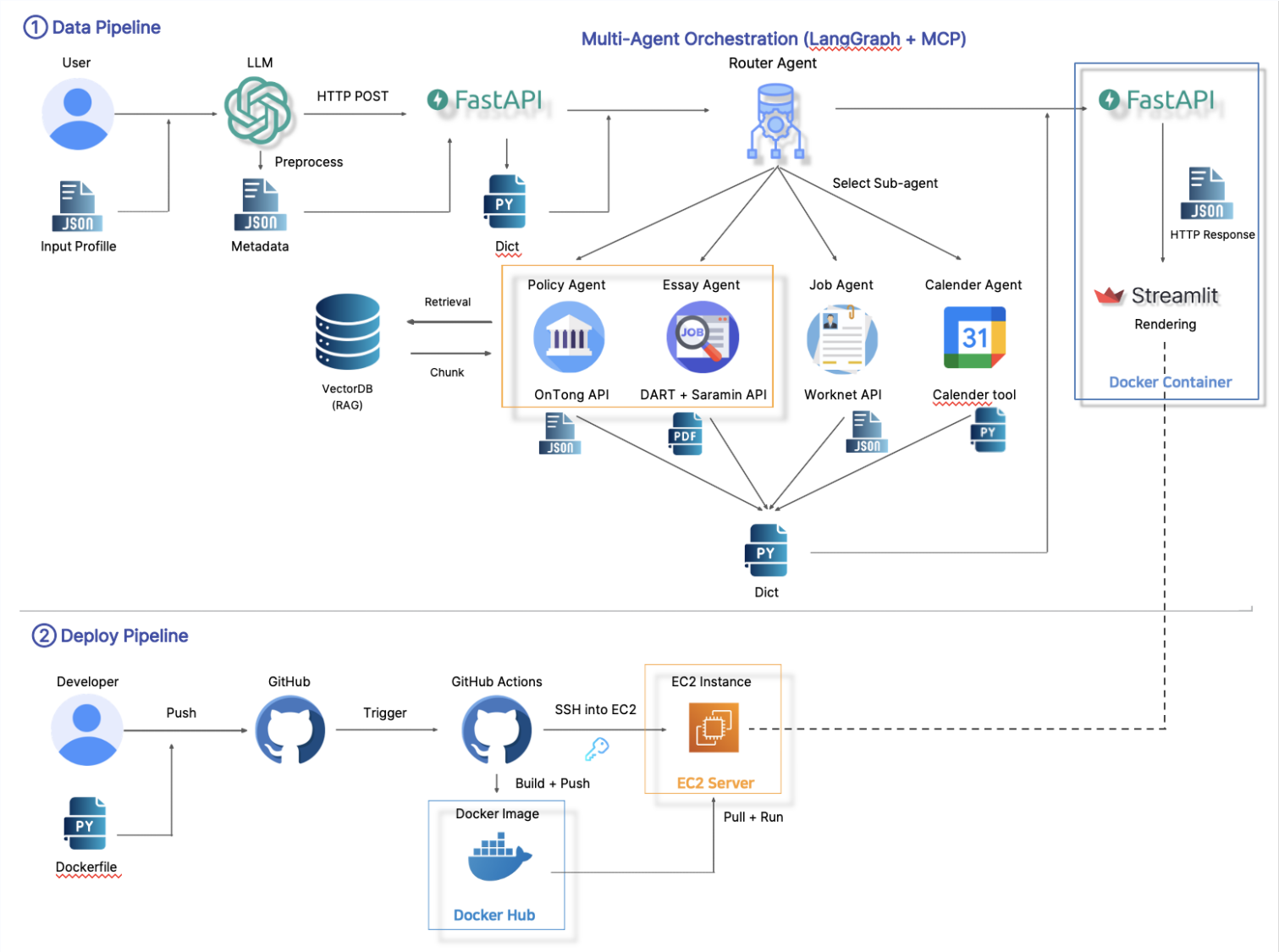
Gives role-aware feedback
and improvement suggestions

4. Calendar

Connects policy and hiring deadlines
into the user schedule

Turn scattered opportunities into Single Integrated Workflow

System Architecture



Agent role design

Router Agent

- Analyzes User Request
- Calls needed Specialist Agent

Policy Agent

Input: Profile, Region, Income

Process: MCP(OnTong API)
→ Static RAG
→ Eligibility & Benefit Matching

Output: Applicable Programs

Job Agent

Input: Target Role, Experience

Process:
MCP(Worknet + Saramin API)
→ Posting Filtering

Output: Recommended Openings

Essay Agent

Input: Target Company, Resume Draft

Process: Rubric-based Feedback

Output:
Resume Guideline

Calendar Agent

Input: Policy, Hiring Deadlines

Process: Conflict Checking,
Alert Scheduling

Output: Calendar Events and Alerts

Each agent can run independently, while results are integrated around the user goal.

Why a RAG and MCP-based Approach Needed

Project Condition	Technical Choice	Expected Effect
Constantly Changing Policy & Job Data	RAG-based runtime retrieval	Grounded Answers with Current Sources
General LLM gives only vague answers	Static + Dynamic RAG	Citation-backed Responses
External APIs & Tools Requirements	MCP / Tool Calling	Connect Public Data & Calendars
Limited Credits & Training Cost	Lightweight Orchestration	Fast Prototype w/o Retraining

Solve the problem by connecting current data and tools, not by retraining a model.

Dataset and Processing Strategy

Agent Name	Data Content	Data Source	Processing Strategy
Policy Agent	Youth Center API Policy Database	Candidate policies Eligibility & benefits	Hybrid Search : MCP API fetches recommended policies -> static RAG detailed criteria
Job Agent	Worknet API Saramin API	Real-time job postings & qualifications	Real-Time Execution : Dynamic filtering and matching
Resume Agent	DART API Saramin API	Biz models, R&D, risks Job titles & schedules	Dynamic RAG + API
Calendar Agent	Agent Outputs	Deadlines and recruitment events	Task Automation

Experiment and Evaluation Plan

Quantitative Evaluation

Required Datasets

Question (Q)
: user query

Ground Truth (GT)
: Ideal Answer

Context (C)
: Retrieved documents

Answer (A)
: LLM-generated response

Evaluation Process

Jeopardy QA
: Auto-generates Q&GT from original texts

System Execution
: LLM retrieves C & generates A

Ragas Framework
: Evaluates datasets mathematically

Core Metrics

Faithfulness
: No hallucination

Context Recall
: Find all necessary info

Context Precision
: Ranking correct info first

Answer Relevance
: Directly answer the query

Qualitative Evaluation

LLM-as-a-Judge

Utilizing GPT-4o-mini to assess
- ✓ Readability
- ✓ Usefulness
- ✓ tone/manner

Human Evaluation

User Satisfaction Survey

Baseline: single LLM answer(Standard prompt) **vs** **YouthPath** (RAG + Agent answer)

Team roles and responsibilities

SR

Policy Agent + Static RAG

HS

Job Agent + DART/Worknet MCP

JW

Resume Agent + Sample RAG

CW

Router Agent + Evaluation

JH

Backend + FE + AWS Deploy

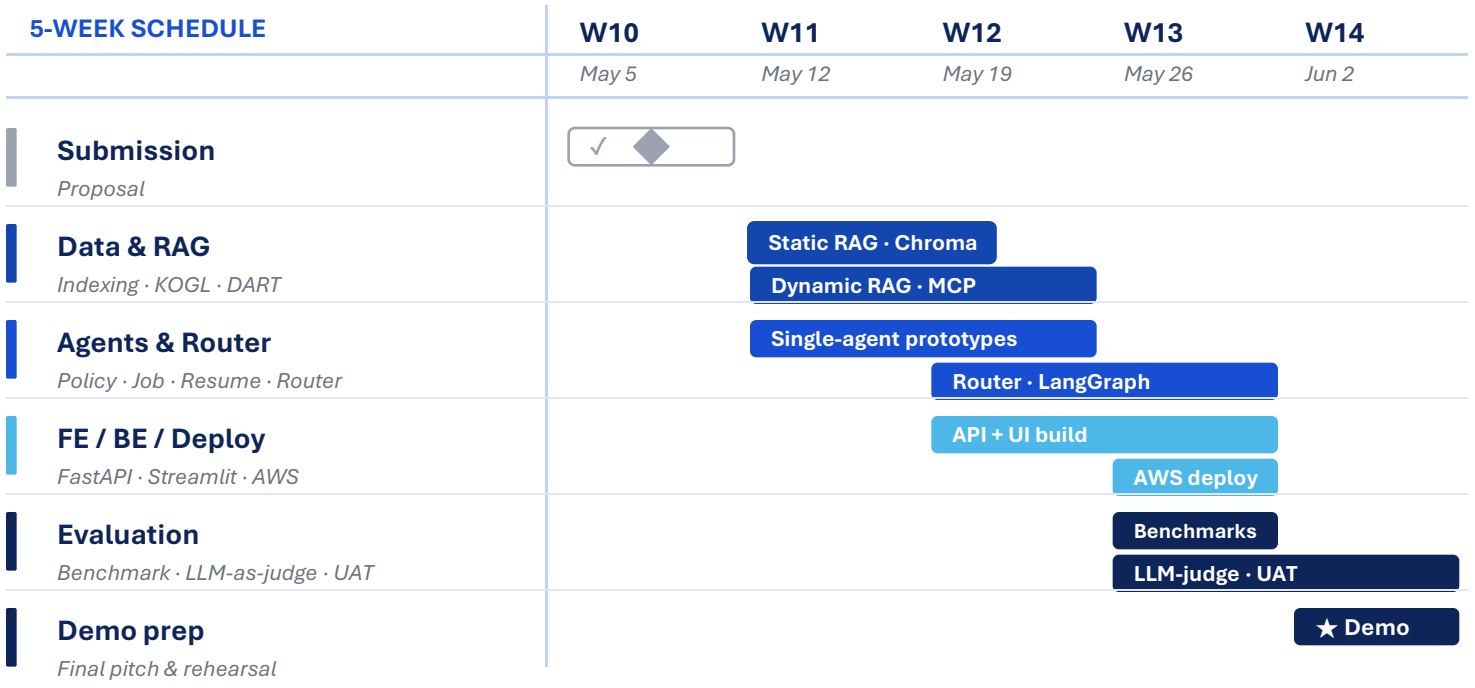
Weekly sync
30-min progress check-in

GitHub
Feature branches

Notion
Shared workspace

Face to face
Real-time chat

Timeline and milestones



Critical path: Static RAG → Single agents → Router integration → FE/BE/Eval → Final demo

COST PLAN

~ $60

estimated runtime cost

LUXIA credit	**Sponsored**
GPT-4o-mini	**~$10**
Gemini Flash	**Free tier**
AWS Academy	**Free credits**

Thank you.